

TLDR

Five API requests below. Four of them are pure additive public accessors over state you already compute internally — zero behavior change. One (Ask 2) is a small new virtual method gated on read-only non-encrypted files, with a `return nullptr` default, so anyone who doesn't opt in sees no difference.

Since it is a decently quick PR, we can submit it if you guys need it.

1. Five small public getters

We need to walk the segment tree from the extension side and read compression metadata. Today these are private or protected. We've been carrying a local patch on our DuckDB submodule fork that just exposes them. It works, but it means we have to ship a fork.

Five getters, all one-liners:

Header	Accessor	Exposes
storage/data_table.hpp	GetRowGroupCollectionRef()	<code>shared_ptr<RowGroupCollection></code> & (today private <code>row_groups</code>)
storage/table/ row_group_collection.hpp	GetRowGroupsDirect()	the segment tree (today's <code>GetRowGroups()</code> is protected)
storage/table/ row_group.hpp	GetColumnDirect(StorageIndex)	<code>ColumnData&</code> (today's <code>GetColumn</code> is protected)
storage/table/ column_segment.hpp	GetCompressionType()	wraps <code>function.get().type</code>
storage/table/ standard_column_data.hpp	GetValidityData()	the private validity <code>ColumnData</code>

Combined sketch:

```
// data_table.hpp
shared_ptr<RowGroupCollection>& GetRowGroupCollectionRef() { return row_groups; }

// row_group_collection.hpp
shared_ptr<RowGroupSegmentTree> GetRowGroupsDirect() const { return GetRowGroups(); }

// row_group.hpp
ColumnData& GetColumnDirect(const StorageIndex &c) const { return GetColumn(c); }

// column_segment.hpp
CompressionType GetCompressionType() const { return function.get().type; }

// standard_column_data.hpp
ValidityColumnData& GetValidityData() { return *validity; }
```

If you'd rather we name them differently or shape them slightly differently, totally fine — just let us know.

2. `BlockManager::GetDirectBlockPointer(block_id)` for read-only files

This is the one that's worth talking through.

`BufferManager::Pin()` is the dominant CPU cost in our scan path. On a column-heavy query, the GPU sits idle waiting for `Pin()` to hand out pointers. The mutex is part of it but not the main cost — most of the time goes to copying the 256KB block into a `FileBuffer` and maintaining the refcount. Both are necessary for writable DBs. For a read-only `.duckdb` file, the block is already in the OS page cache, nothing can evict it, and the mutex is just serializing parallel readers for no reason.

What we're doing right now: `mmap`'ing the file ourselves inside the extension and computing block addresses arithmetically:

```
data_ptr = mmap_base
    + BLOCK_START           // = 4096 * 3 (file-header constant)
    + block_id * alloc_size // from segment.block->GetBlockAllocSize()
    + hdr_size              // from segment.block->GetBlockHeaderSize()
    + block_offset
```

Validated once at startup by `memcmp`'ing the first pinned block against our derived pointer. Gated on read-only + non-encrypted. It works and matches a patched DuckDB to within 2-3% on our benchmarks.

The reason we'd still rather you owned this:

1. The `4096 * 3` constant is yours, not ours. We're reverse-engineering a storage-format invariant.
2. Our `memcmp` validator is self-referential — both sides of the compare came from the same pin, so the check only catches a constant that's grossly wrong, not a subtle layout shift.
3. If this lives in `SingleFileBlockManager` it tracks any future format change automatically. If it lives in our extension, format drift breaks us silently and we don't notice until results go bad.

What we'd PR:

```
// duckdb/storage/block_manager.hpp
class BlockManager {
    virtual data_ptr_t GetDirectBlockPointer(block_id_t block_id) {
        return nullptr; // default: caller falls back to Pin()
    }
};

// SingleFileBlockManager override:
data_ptr_t SingleFileBlockManager::GetDirectBlockPointer(block_id_t block_id) {
    if (!options.read_only || encryption_enabled) return nullptr;
    // lazy-mmap the file with atomic DCL init,
    // return base + BLOCK_START + block_id * alloc_size + header_size
}
```

About 150 LOC across 5 files.

What we're not asking for:

- No changes to `Pin()`. That's the hot write path, hands off.
- No buffer-pool bypass for writable DBs. The read-only gate is the whole safety story.
- No encryption support — `encryption_enabled` short-circuits to null and we fall back to `Pin()`.

3. Expose where block 0 lives in the file

Even if #2 lands, our fallback (encrypted DBs, writable DBs, older DuckDB versions) needs to know where block 0 starts in the file. That constant is the three headers you write at the top of a `.duckdb` — currently $4096 \times 3 = 12288$ bytes. It's a storage-format invariant and only you can authoritatively publish it.

Right now we hardcode 4096×3 and validate against a real pinned block. Same caveat as #2: the validator is self-referential, so it catches a totally wrong constant but not a subtle layout shift.

Either of these works:

```
// duckdb/storage/block_manager.hpp

// Cheapest — public const:
class BlockManager {
    static constexpr idx_t FILE_HEADER_SIZE = 4096 * 3;
};

// More future-proof — accessor in case the offset ever needs to vary:
class BlockManager {
    virtual idx_t GetFileOffsetForBlock(block_id_t block_id) const {
        return FILE_HEADER_SIZE + block_id * GetBlockAllocSize();
    }
};
```

We'd take the `const` if you want minimal surface area, the method if you'd rather encapsulate.

4. Per-column / per-row-group decompressed byte size

We need to know how big a column is going to be once decompressed at two pre-decode points:

1. At query prep, to decide how many row groups to pack into one scan task without OOM-ing the GPU.
2. Just before our string decoder runs, to allocate the output char buffer upfront. Our string decoder is two-pass — Pass 1 computes per-row lengths and exact total chars, Pass 2 gathers. We allocate the output buffer *before* Pass 1 to avoid a sync between the two passes. So Pass 1 needs to know what size buffer to write into.

For both points, today we estimate:

- Fixed-width: `row_count × type_size` — exact, easy.
- VARCHAR: `row_count × max_string_length` from `StringStats` — always an over-estimate, often by 2-4× because `max >> avg`. We over-allocate GPU memory accordingly.

To get the estimate at all, we walk every segment of every projected column of every row group on every query prep. Not free.

You already compute this internally — the CPU scan sizes `DataChunks` from the same state. If you exposed it we'd skip the walk and allocate exactly. The number we want for VARCHAR is the **sum of decompressed string lengths**, not `max × count`. If `StringStats` (or something else) already tracks something closer to the sum, that's enough.

```
// duckdb/storage/table/column_data.hpp
idx_t ColumnData::GetDecompressedSize(idx_t row_start, idx_t row_end) const;

// or per row group:
// duckdb/storage/table/row_group.hpp
idx_t RowGroup::GetColumnDecompressedSize(const StorageIndex &col_idx) const;
```

Either granularity works for us.

5. Publish the compression-format header structs as public POD types

Our GPU decode kernels read the on-disk segment block layout directly. The headers at the start of each segment — `dict_size`, `dict_end`, `bitpacking_width`, FSST symbol table offset, the backwards-stored bitpacking metadata group entries, the RLE count_offset/values/counts layout — are currently internal to DuckDB, so we carry duplicated copies in our CUDA source:

```
// duplicated from DuckDB internals:
struct dict_header_t {
    uint32_t dict_size;
    uint32_t dict_end;
    uint32_t index_buffer_offset;
    uint32_t index_buffer_count;
    uint32_t bitpacking_width;
};

struct fsst_header_t {
    uint32_t dict_size;
    uint32_t dict_end;
    uint32_t bitpacking_width;
    uint32_t fsst_symbol_table_offset;
};
```

We reverse-engineered the layouts from your source and treat them as a contract. There's no version check — we just trust nothing has changed.

This is the ask most likely to prevent silent correctness bugs. We already hit one: our int64 bitpacking unpack for the edge case where a single value spans 3 consecutive 32-bit words was wrong for certain width/offset combinations. Symptom was silent bad answers on timestamp columns. Took a day to track down because the data looked right almost all the time. If those layout structs were public, any DuckDB engineer looking at them would know they have downstream consumers and would think twice before changing them silently.

Suggested shape:

```
// duckdb/storage/compression/storage_format.hpp - new public header
namespace duckdb::storage_format {

constexpr uint32_t STORAGE_FORMAT_VERSION = 1; // bump on on-disk change

struct dictionary_compression_header_t {
    uint32_t dict_size;
    uint32_t dict_end;
    uint32_t index_buffer_offset;
    uint32_t index_buffer_count;
    uint32_t bitpacking_width;
};

struct fsst_compression_header_t {
    uint32_t dict_size;
    uint32_t dict_end;
    uint32_t bitpacking_width;
    uint32_t fsst_symbol_table_offset;
};
```

```
// Bitpacking per-2048-row metadata group:
//   - first 8 bytes of segment = end-of-metadata offset (uint64)
//   - metadata entries are consecutive uint32s, stored backwards from there
//   - each entry: (mode << 24) | (data_offset & 0x00FFFFFF)
//
// RLE segment layout:
//   - first 8 bytes = rle_count_offset (uint64)
//   - values: T[entry_count] starting at byte 8
//   - counts: uint16_t[entry_count] starting at rle_count_offset

} // namespace duckdb::storage_format
```

Consumers can `static_assert` on `STORAGE_FORMAT_VERSION` to fail loudly at compile time when you bump it. Piecemeal is fine — even just the dict and FSST headers as public POD types would cover our two most-used codecs.

tldr

#	What	Cost on your side	What it kills on ours
1	Five public getters	five one-liners across five headers	Need to ship a DuckDB fork
2	<code>GetDirectBlockPointer</code>	~150 LOC, 5 files, read-only gate	The dominant CPU cost in our scan
3	Block-0 file offset	1-3 lines	Hardcoded <code>4096*3</code> + a self-referential validator
4	Decompressed byte size	one accessor over existing internal state	Per-query segment walk + 2-4× VARCHAR over-allocation
5	Public format headers	new header file with POD structs	Silent-correctness-bug class (we hit one)

Asks 1, 3, 4, 5 are all pure additive public accessors — nothing existing changes. Ask 2 has real code but is gated to read-only + non-encrypted with a `return nullptr` default, so existing callers see nothing.

Happy to drive the PRs. Let us know what you think.